

Physics-Aware and Execution-Verifiable Post-Training for Language-Driven Optimal Power Flow Model Adaptation

Yifan Zhang and Coauthors to Be Confirmed

Abstract—Power-system studies repeatedly adapt optimal power flow (OPF) models to updated forecasts, equipment availability, operating limits, and security or policy requirements. A large language model (LLM) can assist this process, but regenerating a complete OPF program is both unnecessary and error-prone: syntactically valid code may modify the wrong grid object, perform a silent no-op, or create a constraint that is never attached to the optimization model. This paper formulates language-driven OPF modeling as a constrained model-adaptation task. A validated Python/Egret backbone remains fixed, while the LLM produces a machine-readable specification, a parameter-modification hook, and a structural-extension hook. We construct 4,400 training and development examples and a separately rebuilt 123-task benchmark covering explicit and scenario-inferred parameter changes with and without structural extensions. All records pass parameter-target and post-state auditing, while the structural verifier is validated across nine extension families and targeted counterexamples. A cumulative curriculum teaches the four adaptation capabilities, and we propose a three-stage Physics-Aware Power-Reward that combines interface validity, realized grid/model correctness, and execution/solver feedback. The deterministic components currently support data auditing and strict candidate evaluation and are designed for integration into the final reinforcement-learning reward. Preliminary experiments on the pre-freeze data version show that supervised fine-tuning substantially improves executable generation, especially with demonstrations. **[TODO: Replace the preliminary statement with final ProOPF-DP-v11/ProOPF-BP-v6 SFT and RL results.]**

Index Terms—Large language models, optimal power flow, power-system modeling, code generation, supervised fine-tuning, reinforcement learning with verifiable rewards.

I. INTRODUCTION

OPTIMAL power flow (OPF) is a foundational tool for coordinating generation and network operation subject to power-balance equations, equipment capabilities, voltage limits, and transmission constraints [1], [2]. Its mathematical core is stable, but the instance and sometimes the formulation must be adapted as forecasts, availability, operating rules, and study objectives change. High renewable penetration increases the range and diversity of operating scenarios that must be studied. By April 2024, China’s installed wind and photovoltaic capacity had exceeded 1.1 TW, 38% above the previous year [3]. Forecast errors affect net load, ramping needs, and reserve headroom, while planned or forced outages change

network availability and the feasible region of AC and DC OPF.

The resulting model changes have two recurring forms. First, an operating study may update data already present in a canonical formulation, such as active and reactive demand, generator limits and costs, line impedance and ratings, transformer taps, or equipment status. Second, the study may alter the algebraic structure by introducing reserve variables, emission terms, contingency constraints, or corrective-action logic. The former are *parameter modifications*; the latter are *structural extensions*. This distinction reflects practical study preparation: a forecast update changes the model state, whereas a new reserve or security requirement changes what the model must optimize or enforce.

Modern operating environments place pressure on the conventional chain of expert interpretation, mathematical reformulation, software implementation, debugging, and solver execution. California ISO, for example, operates real-time dispatch intervals of 15, 5, and in specific conditions 1 minute, while PJM updates dispatch rates on a 15-second cycle [4], [5]. These intervals do not imply that engineers rewrite an OPF formulation at every dispatch step. Rather, they illustrate the time-sensitive environment in which new scenarios, outages, and rules must be converted into validated studies. Scale further increases the maintenance burden: a 2024 Southern regional market trial reported more than 6,000 nodes, 4,500 lines, 1,800 generating units, and 1.5 million constraints [6].

LLMs offer a natural-language interface for assisting this modeling workflow. Instead of directly controlling the grid or replacing the numerical optimizer, an LLM can translate a dispatch, maintenance, or planning instruction into a small change to an existing model. This role is narrower than full-program synthesis and better aligned with established engineering practice: trusted code retains the standard network equations and solver interface, while generated content is limited to the scenario-dependent difference.

The central obstacle is that ordinary software checks do not establish electrical or modeling correctness. A response may parse as JSON and Python yet refer to a nonexistent branch, confuse a generator identifier with its bus, edit a reactive-power quantity in a DC model where it has no effect, or leave a requested change unapplied. A structural hook may also construct a detached Pyomo expression or omit one contingency while the complete program still executes and converges. Even the final objective value is insufficient: a wrong edit can leave the same active set, a nonbinding constraint can preserve the

Y. Zhang is with the School of Advanced Manufacturing and Robotics, Peking University, Beijing, China (e-mail: 2601112747@stu.pku.edu.cn).

The complete author list, author order, corresponding author, affiliations, and funding statement are to be confirmed before submission.

cost, and a silent no-op can accidentally match the reference solution. We use *physical hallucination* in a testable sense to denote such a mismatch between the requested grid/model change and the realized data, algebraic model, or physically constrained solution.

Existing natural-language optimization datasets demonstrate that post-training can improve formulation and code generation [7]–[10]. However, they do not explicitly target the joint parameter–structure dynamics of OPF or verify object identity, AC/DC applicability, and attachment of generated model components. ProOPF introduced a difference-based taxonomy and an expert benchmark for professional-grade power-system optimization modeling [11]. Building on that problem representation, this work asks a narrower question: *can a compact open-weight model be post-trained to generate executable OPF adaptations whose intended grid and model effects can be verified deterministically?*

We address these failures with three components. A fixed Python/Egret backbone reduces the amount of generated code and exposes two controlled insertion points. An audited dataset teaches the response contract and supplies trusted post-state and structural oracles. Curriculum supervised fine-tuning (SFT) then introduces explicit parameter changes, scenario inference, and structural extensions in stages. Finally, a gated Physics-Aware Power-Reward is designed to provide dense feedback without allowing a plausible objective value to compensate for an incorrect grid object or unattached constraint.

The main contributions are as follows.

- We operationalize ProOPF’s difference-based representation as a structured specification and two controlled Python/Egret hooks around a validated AC/DC OPF backbone. This interface exposes the realized data and algebraic-model changes to deterministic inspection while preserving standard network equations.
- We construct a Python/Egret post-training corpus with 4,000 training and 400 source-separated development examples, and separately rebuild a 123-task four-level benchmark. All records are checked against real equipment and complete parameter post-states; trusted structural hooks are solved, and a nine-family structural verifier is tested against positive and adversarial cases.
- We propose cumulative curriculum SFT and a three-stage Physics-Aware reward that separates interface validity, realized grid/model correctness, and solver outcome. The parameter and structural verifiers already support auditing and strict evaluation; integration into the final RL implementation remains an explicit experiment in this V1 draft.
- We establish a diagnostic evaluation protocol that reports the complete correctness funnel rather than objective accuracy alone, thereby exposing whether failures arise from formatting, object/parameter semantics, model assembly, execution, or the OPF solution. **[TODO: Update this contribution with final SFT/RL, ablation, and case-study findings.]**

The remainder of the paper is organized as follows. Section II reviews related work. Section III formulates the adaptation task and its correctness criteria. Section IV describes data construction and audit. Section V presents curriculum SFT

and the Physics-Aware reward. Sections VI and VII describe the experimental protocol and current results, respectively. Section VIII discusses operational scope and limitations.

II. RELATED WORK

A. Natural-Language Optimization Modeling

Natural-language optimization modeling has progressed from translating descriptions into structured formulations to generating solver-ready code. NL4Opt established a benchmark for formulating optimization problems from language [7]. MAMO, OptiBench, Optimus, and ORLM expanded task coverage, solver integration, data synthesis, or model specialization [8]–[10], [12]. These works demonstrate that domain data and executable feedback are valuable, but most tasks emphasize heterogeneous operations-research formulations. They do not explicitly verify whether code modifies a particular grid device, whether a quantity is active under an AC/DC formulation, or whether a generated constraint is attached to an existing OPF model.

B. LLMs for Power-System Analysis and Modeling

LLMs have been studied as interfaces to power-system simulation tools and code-generation environments. Jia *et al.* combine retrieval, reasoning, and execution feedback in a multi-agent framework for power-system simulation [13]. Zhang *et al.* study LLM-based adaptive OPF programming for distribution networks [14]. ProOPF provides 12,000 data instances, 121 expert-annotated benchmark tasks, a difference-based four-level taxonomy, and SFT experiments [11]. The present work therefore does not claim the taxonomy itself as new; it rebuilds the task in Python/Egret and adds source-separated development data, realized post-state and structural verification, curriculum training, and a three-stage reward design. A recent physics-guided feeder-generation framework uses SFT, GRPO, and gated physical rewards to synthesize complete distribution feeders from textual requirements [15]. Its reward checks properties of a generated network instance, whereas ours adapts an existing AC/DC OPF and checks the requested object, complete data post-state, and attached algebraic components.

C. External Verification and Verifiable Rewards

Formal and executable domains can provide objective feedback unavailable from human preference labels alone. Proof assistants such as Lean verify generated mathematical proofs [16], [17], while interpreters and solvers can validate programs and optimization models. Solver-informed reinforcement learning uses general optimization solvers and objective/model signals to ground optimization modeling [18]. In OPF adaptation, however, convergence or an objective match does not prove that the requested device was modified or that a new constraint was attached. The proposed reward therefore places power-grid object, complete post-state, and structural checks before solver feedback. Inference-time diagnosis and repair frameworks are complementary: they can repair residual failures after generation, while this work focuses on internalizing adaptation capability during post-training.

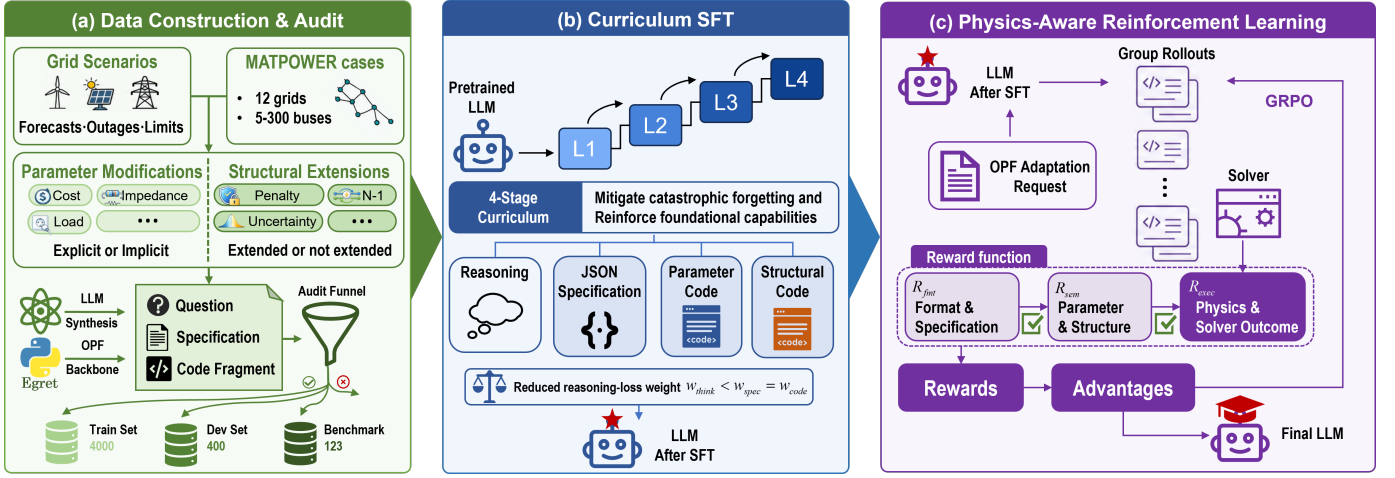


Fig. 1. Draft overview of the proposed workflow: audited construction of parameter-modification and structural-extension data, cumulative curriculum SFT, and physics-aware reinforcement learning with gated rewards and solver feedback.

III. PROBLEM FORMULATION AND FRAMEWORK

Figure 1 summarizes the full workflow before the individual task and verifier definitions are introduced.

A. Canonical OPF and Adaptation Operators

Let d collect the network data—loads, generator attributes, branch parameters, limits, costs, and status—and let z collect the decision variables. A canonical AC or DC OPF backbone can be written abstractly as

$$\mathcal{Q}_{\text{base}}(d) : \min_z F_0(z; d) \quad (1a)$$

$$\text{s.t. } H_0(z; d) = 0, \quad (1b)$$

$$G_0(z; d) \leq 0, \quad (1c)$$

where H_0 contains the formulation-specific power-balance equations and G_0 contains operating bounds and network limits. Standard code for data loading, AC/DC model construction, and solver invocation is treated as trusted infrastructure.

An instruction x may imply a parameter transformation T_p and structural additions $(\Delta F, \Delta H, \Delta G)$. The intended adapted problem is

$$d' = T_p(d; x), \quad (2a)$$

$$\min_z F_0(z; d') + \Delta F(z; d', x) \quad (2b)$$

$$\text{s.t. } H_0(z; d') = 0, \quad \Delta H(z; d', x) = 0, \quad (2c)$$

$$G_0(z; d') \leq 0, \quad \Delta G(z; d', x) \leq 0. \quad (2d)$$

Examples of T_p include changing a load forecast, derating a line, or taking a generator out of service. Examples of structural additions include reserve variables and headroom constraints, an emission cap, an N-1 security constraint family, or a voltage-deviation term in the objective.

B. Difference-Based LLM Interface

Rather than ask the LLM to regenerate (1), we define

$$(\hat{S}, \hat{c}_p, \hat{c}_s) = f_\theta(x), \quad \hat{\mathcal{Q}} = \text{Assemble}(\mathcal{Q}_{\text{base}}; \hat{c}_p, \hat{c}_s), \quad (3)$$

TABLE I
FOUR-LEVEL OPF ADAPTATION HIERARCHY

Level	Parameter values	Structure	Representative demand
L1	Explicit	Retained	Direct data edit
L2	Scenario-inferred	Retained	Infer object/direction
L3	Explicit	Extended	Data edit plus model term
L4	Scenario-inferred	Extended	Joint inference and extension

where $\hat{S}$ is a JSON specification, $\hat{c}_p$ is a Python parameter hook applied to Egret's model data before construction, and $\hat{c}_s$ is a structural hook applied to the Pyomo model after construction. The structural hook contains `pass` when the base structure is retained. The answer contract contains four regions: a concise rationale, the JSON specification, the parameter hook, and the structural hook. The rationale is supervised with reduced weight and is not trusted as evidence of correctness.

The two insertion points provide both modularity and observability. Parameter code can be checked by comparing the complete post-modification data state with a trusted reference. Structural code can be checked after model assembly by inspecting objectives, variables, and constraints. Generated code is never allowed to replace the solver driver or the standard power-flow equations.

C. Four-Level Task Hierarchy

The task hierarchy crosses two axes: whether numerical changes are explicit or must be inferred from an operating scenario, and whether the canonical formulation is retained or extended. Table I summarizes the four combinations.

L1 and L3 are evaluated with the values stated in the request. L2 and L4 contain two held-out runtime instantiations, labeled mild and severe, that are not exposed as numerical values in the question. Both instantiations must pass. This prevents a model from hard-coding one value while preserving the required direction and object identity.

D. Verifiable Correctness

Let I_{fmt} indicate a valid response contract, JSON schema, function signature, and Python syntax. Let I_{spec} indicate agreement of the case, formulation, grid objects, operations or directions, structural family, and solver requirements. Parameter correctness is defined by equality of the candidate and trusted post-states,

$$I_p = \mathbb{I}\left[T_{\hat{c}_p}(d; \xi) = T_{c_p^*}(d; \xi)\right], \quad (4)$$

where ξ is empty for explicit tasks and denotes a runtime strategy for inferred tasks. Structural correctness is

$$I_s = \mathbb{I}\left[\Phi(\hat{Q}) = \Phi(Q^*)\right], \quad (5)$$

where Φ is a semantic signature of the objective delta, added variables and constraints, and protected base-model components. In the current candidate evaluator, I_{solve} requires successful solver execution and reference-objective agreement. The final protocol will additionally record solver termination and explicit feasibility residuals. The strict end-to-end indicator is

$$I_{\text{strict}} = I_{\text{fmt}} I_{\text{spec}} I_p I_s I_{\text{solve}}. \quad (6)$$

This conjunction is deliberately stronger than code execution or objective agreement alone.

IV. EXECUTABLE DATASET CONSTRUCTION

A. Python/Egret Backbone and Scope

Egret is an open-source Python package built on Pyomo for electric-grid optimization [19], [20]; MATPOWER cases provide the public network instances [21]. Egret’s separation between model data and the algebraic Pyomo model naturally exposes the parameter and structural insertion points in (3). The current scope includes continuous ACOPF and DCOPF adaptations. It does not cover unit commitment, transient simulation, protection logic, or autonomous online control.

The semantic release candidate uses 12 MATPOWER systems ranging from 5 to 300 buses: *case5*, *case6ww*, *case9*, *case14*, *case18*, *case24_ieee_rts*, *case30*, *case39*, *case57*, *case60nordic*, *case118*, and *case300*. Parameter changes cover active/reactive demand, voltage and generation limits, generation-cost coefficients, branch resistance/reactance/susceptance and ratings, transformer taps, and equipment status. Structural examples span the nine families listed in Table II. All L1/L2 records use ACOPF. In train and development data, each of L3 and L4 contains 425 ACOPF and 775 DCOPF records; the benchmark contains 11 ACOPF records in each structural level and 32 DCOPF records in total.

B. Synthesis and Benchmark Rebuilding

We retain the task taxonomy and scenario concepts of ProOPF but do not mechanically translate ProOPF-D into Python. The original MATLAB corpus cannot directly supervise Egret/Pyomo APIs or provide the realized post-state and structural signatures required here. Training specifications and Python/Egret hooks are therefore synthesized and

TABLE II
STRUCTURAL-EXTENSION COVERAGE IN TRAIN AND DEVELOPMENT DATA

Structural family	Examples
Angle-difference penalty	258
Voltage-deviation penalty	292
Reactive-generation cost	276
N-1 contingency constraints	266
Spinning-reserve requirement	246
Regulation-up requirement	254
Quadratic-emission cost	260
CO ₂ emission cap	266
Reactive-reserve requirement	282
Total L3/L4	2400

executed independently. The 123 benchmark questions are separately rebuilt from ProOPF-B scenarios, with corrections and revalidation where the Python interface or physical semantics differ from the original MATLAB implementation. All BP-v6 changes were fixed before evaluating any model trained on DP-v11. **[TODO: Document the exact template/API/inherited-text provenance and quantify the ProOPF-B tasks retained, repaired, replaced, and added when the public release history is finalized.]**

For each candidate task, the construction pipeline selects a real case and formulation, samples valid parameter targets and values or a scenario-direction pair, and optionally selects a structural family with its hyperparameters. A trusted parameter hook and structural hook are then assembled around the backbone. Explicit tasks are solved once; inferred tasks are solved under both runtime strategies. Only tasks that pass the applicable construction checks are retained. Natural-language questions describe the operating request without exposing internal level names, runtime placeholders, or held-out strategy values.

C. Deterministic Audit Pipeline

The pipeline is stricter than “program runs and objective matches,” but the construction-time and generated-answer checks have different coverage in V1.

- 1) *Interface and grid semantics*: required fields and hook syntax are validated; the case and AC/DC formulation must be valid; every target must resolve to the intended real bus, generator, load, or branch; and inactive quantities are excluded from incompatible formulations.
- 2) *Parameter post-state*: every retained hook is executed in a fresh subprocess, and the complete typed Egret `ModelData.data` tree is compared with the trusted post-state. This detects no-ops, wrong identifiers, wrong values, and collateral edits.
- 3) *Structural semantics*: trusted gold hooks are produced from controlled nine-family templates and paired with solved labels. For generated answers, candidate and trusted hooks are applied to separately constructed models; polynomial objective deltas and supported constraints are canonicalized so component names and term order do not affect equivalence. The verifier’s nine positive families and targeted no-op, missing-term, wrong-

Algorithm 1 Strict Evaluation of One Generated OPF Adaptation

Require: Request x , specification $\mathcal{S}$, hooks (c_p, c_s) , pristine case d

- 1: Parse response contract, $\mathcal{S}$, and Python AST
 - 2: Resolve the case, formulation, and all grid objects
 - 3: **for** each runtime strategy ξ (one for explicit tasks, two for inferred tasks) **do**
 - 4: Run c_p in a fresh subprocess and snapshot the post-state
 - 5: Compare the post-state with the trusted parameter oracle
 - 6: Build fresh candidate and trusted Egret/Pyomo models
 - 7: Run c_s and compare semantic structural signatures
 - 8: Execute the assembled OPF and collect execution status and objective
 - 9: Reject if any required check fails
 - 10: **end for**
 - 11: **return** Strict decision and per-gate report
-

coefficient, and base-tampering counterexamples pass regression tests. A full per-record structural re-audit of all 4523 gold records has not been claimed.

- 4) *Execution and objective*: benchmark gold programs must return an objective and match the reference within an absolute tolerance of 10^{-4} or relative tolerance of 10^{-3} . Explicit termination-condition and feasibility-residual logging remain to be added to the final evaluator.

The stated objective tolerance records the current software setting; it is not used as a substitute for semantic verification.

[TODO: Before final evaluation, preregister the tolerance and report a tighter-tolerance sensitivity analysis.]

Hook execution uses a fresh subprocess with a timeout, and the caller handles serialized reports. This isolates parent-process Python state, but it is not an operating-system security sandbox: filesystem, network, and resource isolation remain deployment requirements.

Algorithm 1 summarizes the intended strict evaluation of one generated adaptation.

The audit exposed errors that objective-only filtering would miss. For example, CO₂ caps were originally expressed in MW but compared directly with per-unit generation variables. The corrected hook divides the cap by `baseMVA`; 266 records and 399 stored results were regenerated. Similarly, coefficient-based generator terms now store generator identifiers in addition to bus numbers, avoiding ambiguity when multiple generators share a bus.

D. Source-Separated Split and Statistics

Table III reports the frozen candidate split. Development examples are sampled by semantic lineage: an explicit task and its inferred counterpart remain together, as do structurally retained and extended variants derived from the same source scenario. No lineage crosses train and development. The benchmark is separately rebuilt and has zero overlap with the training/development data under full-specification and parameter-identity signatures.

TABLE III
FROZEN CANDIDATE DATA SPLIT

Split	L1	L2	L3	L4	Total
Train	900	900	1100	1100	4000
Development	100	100	100	100	400
Benchmark	36	33	28	26	123

Across all single-level train, development, and benchmark records, 4523/4523 records pass the parameter-target and post-state audit, requiring 6782 hook executions. All 64 explicit benchmark gold programs solve and reproduce their stored objectives. All 59 inferred benchmark programs pass both strategies, for 118/118 successful strategy solves. The parameter/structural verifiers, strict evaluator, and data converter pass 34 targeted regression tests, including no-op, wrong-object, missing-constraint, wrong-coefficient, and base-model-tampering counterexamples. The benchmark covers six of the nine training structural families; emission-cost, emission-cap, and reactive-reserve extensions remain train/development-only and therefore cannot support test-set generalization claims in V1.

Two runtime strategies need not produce different objective values: this occurs in 202 L2 and 436 L4 train/development records and in 12 L2 and 9 L4 benchmark records, although their audited post-states differ. These records are retained because a nonbinding limit or unchanged active set can make the equality physically legitimate. They also provide direct evidence that objective-only evaluation cannot identify the intended modification and cannot support a monotonic “mild versus severe” claim by itself.

The remaining label decision concerns the auxiliary rationale: 49 train/development responses retain legacy mentions of the internal level taxonomy. User questions, prompts, and benchmark responses contain no such hints. **[TODO: Before final retraining, either rewrite and fact-audit these rationales once or document their retention; do not change task identities, hooks, results, or splits.]**

V. PHYSICS-AWARE POST-TRAINING

A. Cumulative Curriculum SFT

The two hierarchy axes create a natural curriculum. Stage 1 learns the response contract and explicit data edits from L1. Stage 2 adds scenario-to-parameter inference through L2. Stage 3 introduces Pyomo structural construction through L3. Stage 4 combines scenario inference and structural extension through L4. Earlier data are retained in every later stage, yielding cumulative training sizes of 900, 1800, 2900, and 4000 and development sizes of 100, 200, 300, and 400. Replay is important because training only on a new level previously caused the model to invent structural extensions for simpler requests.

For a target response $y = (y_1, \dots, y_T)$, SFT minimizes a token-weighted negative log likelihood,

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{t=1}^T w_t \log p_{\theta}(y_t \mid x, y_{<t}), \quad (7)$$

where specification and code tokens receive unit weight and rationale tokens receive $\lambda_r < 1$. The preliminary configuration uses $\lambda_r = 0.1$. This choice treats reasoning as soft guidance rather than a fixed template while concentrating supervision on the executable interface. **[TODO: Confirm λ_r through an ablation on the frozen development set.]**

B. Three-Stage Physics-Aware Reward

After SFT, reinforcement learning targets failures that token likelihood does not directly distinguish. For a sampled answer y , the proposed reward is

$$R(y) = \alpha_f r_{\text{fmt}} + g_1 \alpha_s r_{\text{sem}} + g_1 g_2 \alpha_e r_{\text{exec}}, \quad (8)$$

where r_{sem} is the realized grid/model-correctness stage, g_1 and g_2 indicate complete passage of the preceding gate, and $\alpha_f, \alpha_s, \alpha_e$ are stage weights.

The three rewards are defined as follows.

- r_{fmt} checks the four response regions, JSON schema, required functions, and Python syntax. It provides inexpensive feedback before any generated code is executed.
- r_{sem} checks the case, AC/DC formulation, real grid objects, operations or directions, structural intent, parameter post-state, structural equivalence, and protected base-model invariants. It therefore evaluates the realized grid/model change rather than language semantics alone. The gold structural family, not a candidate-declared family, selects the verifier and prevents reward gaming through self-reported metadata.
- r_{exec} runs the complete adaptation and checks successful solver execution and reference-objective agreement. For L2/L4, both runtime strategies must pass. **[TODO: Extend this stage with termination-condition and feasibility-residual checks before final RL.]**

The gate ordering is intentional. A wrong-device edit cannot earn the solver reward merely because the resulting objective is close. A detached constraint cannot pass the semantic model check even if the unmodified backbone converges. Conversely, equivalent Pyomo expressions with different component names or term order can pass because verification operates on canonicalized model semantics rather than source-text equality.

C. Shared Verification Infrastructure

The object resolver, post-state comparator, structural verifier, and subprocess runner are currently used for auditing and strict candidate evaluation. The final RL reward is designed to call the same components rather than maintain a second definition of correctness. This makes error analysis directly actionable: if the SFT model primarily fails object identity, parameter post-state, or reserve construction on development data, the corresponding reward component can be emphasized without inspecting the held-out benchmark. **[TODO: Wire the frozen verifier API into RL/proopf_reward.py and add reward parity tests before running GRPO.]**

TABLE IV
RL CONFIGURATION TO BE FINALIZED

Item	Draft setting
Initialization	Best Stage-4 SFT checkpoint on dev
Policy optimizer	GRPO
Rollouts per request	[TODO: G]
Learning rate	[TODO: Select on dev]
Sampling temperature	[TODO: Select on dev]
KL coefficient	[TODO: Select on dev]
Reward weights	[TODO: $\alpha_f, \alpha_s, \alpha_e$]
Early stopping	Four-level dev macro strict accuracy
Test access	Once after configuration freeze

D. Group-Relative Policy Optimization

The intended policy optimizer is group relative policy optimization (GRPO) [22]. For each request x , the policy samples a group of G completions with rewards $R_1, \dots, R_G$. A normalized group advantage is

$$A_i = \frac{R_i - \text{mean}(R_1, \dots, R_G)}{\text{std}(R_1, \dots, R_G) + \epsilon}. \quad (9)$$

The clipped policy objective increases the likelihood of above-average verified adaptations while a KL term limits deviation from the SFT reference policy. The final reward weights, rollout count, KL coefficient, temperature, learning rate, and stopping rule must be selected only on the source-separated development set. Table IV records the decisions that remain open in V1.

VI. EXPERIMENTAL PROTOCOL

A. Research Questions

The final experiments are organized around five questions.

- **RQ1:** Does domain SFT improve strict, executable OPF adaptation over the base model and a strong API model?
- **RQ2:** Does the cumulative curriculum retain earlier capabilities better than joint or non-replay training?
- **RQ3:** Which errors remain after SFT, and does Physics-Aware RL improve them without sacrificing simpler levels?
- **RQ4:** Which reward gates and deterministic verifiers are necessary to prevent no-op and objective-matching failures?
- **RQ5:** How does performance vary across AC/DC formulations, system sizes, modified object types, and structural families?

B. Models, Training, and Decoding

The preliminary SFT model starts from Qwen2.5-7B [23]. It is full-parameter fine-tuned in bf16 with FSDP on four NVIDIA A100 GPUs, effective batch size 8, and reasoning-token weight 0.1. Each curriculum boundary restarts the optimizer and learning-rate scheduler. Table V lists the completed pre-freeze run on ProOPF-DP-v10. The final paper must re-train on the source-separated ProOPF-DP-v11 split and select each checkpoint using its cumulative development set.

Baselines include the unmodified Qwen2.5-7B and DeepSeek-V4-Flash. All models receive the same output

TABLE V
COMPLETED PRELIMINARY SFT CONFIGURATION ON DP-v10

Stage	Train Epochs	LR	Max tokens
L1	1000	5	1.0×10^{-5}
L1+L2	2000	2	1.0×10^{-5}
L1+L2+L3	3200	2	7.5×10^{-6}
L1+L2+L3+L4	4400	2	5.0×10^{-6}

contract and use greedy decoding for the reported preliminary comparison. “Zero-demonstration” means that the system prompt specifies the complete response schema but contains no worked answer; “two-demonstration” additionally includes two level-appropriate examples. This terminology avoids implying that the model receives no task instructions.

[TODO: For final experiments, record exact model revisions, tokenizer, PyTorch/Pyomo/Egret/solver versions, GPU hours, random seeds, decoding limits, API access date, and monetary cost. Add at least one current open-weight model and repeat the principal training setting across three seeds if resources permit.]

C. Metrics

The primary metric is the strict accuracy in (6). To diagnose failures, we also report the cumulative pass rates for parsing, semantic specification, parameter post-state, structural verification, execution, and objective agreement. Macro averages across L1–L4 prevent levels with more evaluation tasks from dominating model selection. All checkpoint, hyperparameter, and reward decisions use development data; ProOPF-BP-v6 is reserved for final evaluation.

The preliminary results in Section VII-A were produced before the deterministic semantic gates were integrated. Their historical “strict E2E” definition is only $I_{\text{fnt}} I_{\text{run}} I_{\text{obj}}$ and must not be confused with (6). Its parser also permits limited tag/JSON normalization and fills selected metadata from the task context. These values are retained in this V1 draft to document the evidence already available, not as final frozen-benchmark claims.

VII. RESULTS AND ANALYSIS

A. Preliminary Baseline and SFT Results

Table VI reports the completed comparison on ProOPF-BP-v5. Under the legacy parse–execute–objective metric, the final SFT checkpoint improves Qwen2.5-7B from 0/123 to 15/123 in zero-demonstration evaluation and from 15/123 to 66/123 with two demonstrations. It equals the aggregate DeepSeek result with two demonstrations, although the level distribution differs: SFT is stronger on L2, whereas DeepSeek is stronger on L1, L3, and L4.

The pass-rate funnel in Table VII clarifies the zero-demonstration zeros. DeepSeek produces a parseable response for 91/123 tasks but no executable Egret adaptation. The base Qwen model executes only 2/123 responses and none reaches the objective. Thus, the result reflects an interface and implementation mismatch, not an absence of all OPF

reasoning. With demonstrations, both strong API prompting and SFT greatly improve execution.

The hardest setting is L4, where the model must infer parameter direction and generate a structural extension. L4 two-demonstration parsing reaches 22/26 for the SFT checkpoint, but only 7/26 execute and 5/26 match the objective. This gap motivates semantic and structural reward components rather than further format-only training.

B. Curriculum Retention and Residual Errors

Within the preliminary run, the L3 specialist solves 16/28 L3 two-demonstration tasks under the legacy E2E metric, whereas the final Stage-4 checkpoint solves 10/28. L1 and L2 improve after later stages, but interference between explicit and inferred structural extensions remains. The final experiment will compare cumulative curriculum, one-stage mixed SFT, and curriculum without replay on the same fixed development and test sets.

The pre-freeze error analysis records 100 JSON parse failures among 440 generated answers, 43 Python syntax errors, and 78 failures in implicit runtime-identity mapping. After parsing succeeds, frequent semantic failures include adjacent but incorrect component identifiers, confusion between load and bus indices, unsupported Egret/Pyomo fields, missing structural terms, and structurally valid but numerically incorrect objectives. These categories map directly to the three reward gates.

C. Preliminary RL Evidence

An early L1 prototype contains one archived 36-task few-shot run with 22/36 objective-correct outputs, compared with 13/36 for its preceding SFT checkpoint. A second appended run reached 20/36, and neither run used the current strict protocol. They predate the frozen data, source-separated development set, three-stage deterministic verifier, and current four-level SFT. Moreover, the archived L3 artifacts do not reproduce the improvement cited in earlier digest drafts. Consequently, this paper makes no RL performance claim from those runs and does not compare them with Table VI.

[TODO: Run GRPO from the development-selected ProOPF-DP-v11 SFT checkpoint; report L1–L4 zero-/two-demonstration strict accuracy, component rewards, KL, response length, solver-call rate, wall time, and three-seed uncertainty.]

D. Required Ablations and Final Result Table

Table VIII defines the minimum final comparison. Each dash is an intentionally missing experiment, not a zero. The accompanying ablations will compare joint versus curriculum SFT, replay versus no replay, rationale weight 1.0 versus 0.1, objective-only versus gated reward, removal of the semantic gate, removal of the structural verifier, and removal of solver feedback.

TABLE VI
PRELIMINARY OBJECTIVE-MATCHING E2E ACCURACY $E2E_{obj}$ (%) ON PROPF-BP-V5; THESE VALUES MUST BE RERUN WITH THE FINALIZED STRICT EVALUATOR ON BP-V6

Prompt	Model	L1	L2	L3	L4	All
Two demonstrations	Qwen2.5-7B base	30.56	12.12	0.00	0.00	12.20
	DeepSeek-V4-Flash	97.22	24.24	50.00	34.62	53.66
	Qwen2.5-7B + SFT	80.56	66.67	35.71	19.23	53.66
Zero demonstration	Qwen2.5-7B base	0.00	0.00	0.00	0.00	0.00
	DeepSeek-V4-Flash	0.00	0.00	0.00	0.00	0.00
	Qwen2.5-7B + SFT	5.56	24.24	14.29	3.85	12.20

TABLE VII
PRELIMINARY AGGREGATE RESPONSE FUNNEL (COUNTS OUT OF 123)

Prompt	Model	Parse	Execute	$E2E_{obj}$
Zero	Qwen base	64	2	0
	DeepSeek-V4	91	0	0
	Qwen + SFT	66	24	15
Two-demo	Qwen base	66	23	15
	DeepSeek-V4	123	66	66
	Qwen + SFT	109	79	66

TABLE VIII
FINAL STRICT ACCURACY ON PROPF-BP-V6 (EXPERIMENT PLACEHOLDER)

Model/Training	L1	L2	L3	L4	Macro
<i>Zero demonstration</i>					
Qwen2.5-7B base	-	-	-	-	-
Current open-weight baseline	-	-	-	-	-
Qwen2.5-7B + SFT	-	-	-	-	-
Qwen2.5-7B + SFT + RL	-	-	-	-	-
<i>Two demonstrations</i>					
Qwen2.5-7B base	-	-	-	-	-
Strong API baseline	-	-	-	-	-
Qwen2.5-7B + SFT	-	-	-	-	-
Qwen2.5-7B + SFT + RL	-	-	-	-	-

E. Planned Engineering Case Studies

The final paper will include two traceable cases. The first will show a parameter-only request and report the resolved object, before/after data values, and AC/DC objective. The second will combine parameter changes with a reserve, emission, or N-1 structural extension and report the added Pyomo components and, after the evaluator extension is complete, feasibility residuals. Each case will compare the base model, SFT, and SFT+RL outputs at every verifier gate. This presentation is intended to connect aggregate code-generation accuracy to the actual power-system model being solved.

VIII. DISCUSSION

A. Why Adaptation Rather Than Full Generation?

The fixed-backbone design is not merely a device for shortening the target sequence. It encodes an engineering boundary: standard power-flow equations, data loading, and solver control remain in reviewed software, while the model generates only the changing portion. This reduces duplicated code, makes every generated operation inspectable, and allows the verifier to attribute an error to a parameter or structural

hook. Full generation may remain necessary for genuinely new formulations, but it expands both the learning problem and the safety surface.

B. Operational Interpretation

The present system should be viewed as an offline or human-supervised modeling assistant. A planner or operator supplies a study requirement, the LLM proposes a model difference, deterministic gates reconstruct and solve the candidate, and a human can inspect the compact change and audit report. The current evidence does not justify autonomous real-time dispatch, automatic protection actions, or direct execution against a production energy-management system.

The parameter-structure abstraction nevertheless has an operational basis. Forecast and availability changes naturally map to data updates; reserve, emissions, contingency, and corrective requirements map to additions in the objective or constraint set. Future evaluation should measure not only code correctness but also whether the adapted model represents the intended outage, reserve margin, security limit, or policy requirement and whether it changes the resulting dispatch in the expected physical direction.

C. Safety and Deployment Boundaries

Generated Python is untrusted. It must run in an isolated process with explicit imports, time and resource limits, a read-only case library, and no network or credential access. Passing a deterministic verifier is necessary but not sufficient for deployment: the verifier covers only its supported task and structural families. Unsupported requests should abstain or be routed to expert review rather than forced into the nearest known template.

Inference-time repair is complementary to post-training. A repair system can diagnose residual failures after the proposed model generates a candidate. Conversely, a better trained generator reduces the number and severity of repairs. A controlled comparison should distinguish first-pass accuracy, repair success, additional solver calls, and preservation of initially correct outputs.

D. Limitations and Threats to Validity

First, the 12 public systems and nine training structural families do not cover the diversity of utility models, and the final benchmark contains only six systems and six structural families. Second, the benchmark focuses on continuous

AC/DC OPF; unit commitment, stochastic and robust optimization, transient security, and sequential switching remain outside scope. Third, reference-hook comparison assumes that audited gold implementations are correct, although the multi-stage audits reduce this risk. Fourth, preliminary scores were produced on the pre-freeze dataset, a legacy metric, and a permissive parser/executor that must be replaced by the frozen protocol. Fifth, a single 7B base model cannot establish architecture- or scale-independent conclusions. Finally, solver convergence and objective agreement can still mask requirements not encoded in the specification, which motivates future human review and scenario-level validation.

IX. CONCLUSION

This paper operationalizes language-driven OPF programming as a small, auditable adaptation around a trusted Python/Egret backbone. The task representation separates parameter updates from structural extensions and exposes both for deterministic verification. A source-separated four-level corpus and cumulative curriculum SFT address capability acquisition; the proposed gated Physics-Aware reward is designed to align subsequent RL with interface, realized grid/model, and solver correctness. Existing preliminary results show that domain SFT can substantially improve executable generation, while complex structural adaptation remains the principal bottleneck. **[TODO: Replace the final empirical claim after completing the frozen SFT, RL, ablation, and case-study experiments.]**

ACKNOWLEDGMENT

[TODO: Add funding, data/code availability, and acknowledgments after the author team confirms them.]

REFERENCES

- [1] J. Carpentier, "Contribution à l'étude du dispatching économique," *Bulletin de la Société Française des Électriciens*, vol. 3, no. 8, pp. 431–447, 1962.
- [2] Z. Wang, A. Younesi, M. V. Liu, G. C. Guo, and C. L. Anderson, "AC optimal power flow in power systems with renewable energy integration: A review of formulations and case studies," *IEEE Access*, vol. 11, pp. 102 681–102 712, 2023.
- [3] National Energy Administration of China, "Policy interpretation of the notice on renewable-energy accommodation and high-quality development," [Online]. Available: https://www.nea.gov.cn/2024-06/04/c_1310777103.htm, Jun. 2024, accessed: Aug. 24, 2026.
- [4] California Independent System Operator, "Products and services: Real-time market," [Online]. Available: <https://www.caiso.com/market-operations/products-services>, 2026, accessed: Aug. 24, 2026.
- [5] PJM Interconnection, "Dispatch rates," [Online]. Available: <https://www.pjm.com/markets-and-operations/ops-analysis/dispatch-rates>, 2026, accessed: Aug. 24, 2026.
- [6] National Energy Administration of China, "Ensuring the month-long settlement trial operation of the southern regional electricity market," [Online]. Available: https://www.nea.gov.cn/2024-12/04/c_1310787424.htm, Dec. 2024, accessed: Aug. 24, 2026.
- [7] R. Ramamonjison, T. Yu, R. Li, H. Li, G. Carenini, B. Ghaddar, S. He, M. Mostajabdaveh, A. Banitalebi-Dehkordi, Z. Zhou, and Y. Zhang, "NL4Opt competition: Formulating optimization problems based on their natural language descriptions," in *Proc. NeurIPS 2022 Competitions Track*, ser. Proc. Mach. Learn. Res., vol. 220, 2023, pp. 189–203. [Online]. Available: <https://proceedings.mlr.press/v220/ramamonjison23a.html>
- [8] X. Huang, Q. Shen, Y. Hu, A. Gao, and B. Wang, "LLMs for mathematical modeling: Towards bridging the gap between natural and mathematical languages," *arXiv preprint arXiv:2405.13144*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.13144>
- [9] Z. Yang, Y. Wang, Y. Huang, Z. Guo, W. Shi, X. Han, L. Feng, L. Song, X. Liang, and J. Tang, "OptiBench meets ReSocratic: Measure and improve LLMs for optimization modeling," in *Proc. Int. Conf. Learn. Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=sUVetpkLRs>
- [10] C. Huang, Z. Tang, S. Hu, R. Jiang, X. Zheng, D. Ge, B. Wang, and Z. Wang, "ORLM: A customizable framework in training large models for automated optimization modeling," *arXiv preprint arXiv:2405.17743*, 2024. [Online]. Available: <https://arxiv.org/abs/2405.17743>
- [11] C. Shen, Z. Guo, X. Wan, Z. Yang, Y. Zhang, W. Huang, J. Song, Z. Zhang, and M. Sun, "ProOPF: Benchmarking and improving LLMs for professional-grade power systems optimization modeling," in *Proc. 43rd Int. Conf. Mach. Learn.*, ser. Proc. Mach. Learn. Res., vol. 306, 2026, arXiv:2602.03070. [Online]. Available: <https://arxiv.org/abs/2602.03070>
- [12] A. AhmadiTeshnizi, W. Gao, and M. Udell, "OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models," *arXiv preprint arXiv:2402.10172*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.10172>
- [13] M. Jia, Z. Cui, and G. Hug, "Enhancing LLMs for power system simulations: A feedback-driven multi-agent framework," *IEEE Trans. Smart Grid*, vol. 16, no. 6, pp. 5556–5572, 2025, preprint: arXiv:2411.16707.
- [14] B. Zhang, Z. Yi, Y. Xu, Z. Tu, and S. Dong, "Automatic programming for the optimal power flow in distribution networks: An edge-side adaptive computing approach based on large language models," *CSEE J. Power Energy Syst.*, vol. 12, no. 2, pp. 609–621, 2026.
- [15] Z. Zhou, Y. Li, T. Xu, Y. Guo, Z. Yan, and M.-Y. Chow, "A physics-guided fine-tuned LLM-based framework for customized power distribution system feeder generation," *arXiv preprint arXiv:2607.07237*, 2026. [Online]. Available: <https://arxiv.org/abs/2607.07237>
- [16] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Proc. 28th Int. Conf. Automated Deduction*, ser. Lecture Notes in Computer Science, vol. 12699. Springer, 2021, pp. 625–635.
- [17] H. Ying, Z. Wu, Y. Geng, J. Wang, D. Lin, and K. Chen, "Lean Workbook: A large-scale Lean problem set formalized from natural language math problems," in *Advances in Neural Information Processing Systems*, 2024, datasets and Benchmarks Track. [Online]. Available: <https://arxiv.org/abs/2406.03847>
- [18] Y. Chen, J. Xia, S. Shao, D. Ge, and Y. Ye, "Solver-informed RL: Grounding large language models for authentic optimization modeling," *arXiv preprint arXiv:2505.11792*, 2025. [Online]. Available: <https://arxiv.org/abs/2505.11792>
- [19] M. L. Bynum, A. R. Castillo, B. Knueven, C. D. Laird, and J.-P. Watson, "Egret: Electrical grid research and engineering tools," [Online]. Available: <https://github.com/grid-parity-exchange/Egret>, 2021, accessed: Aug. 24, 2026.
- [20] W. E. Hart, C. D. Laird, J.-P. Watson, D. L. Woodruff, G. A. Hackebeil, B. L. Nicholson, and J. D. Sirola, *Pyomo—Optimization Modeling in Python*, 2nd ed. Cham, Switzerland: Springer, 2017.
- [21] R. D. Zimmerman, C. E. Murillo-Sánchez, and R. J. Thomas, "MATPOWER: Steady-state operations, planning, and analysis tools for power systems research and education," *IEEE Trans. Power Syst.*, vol. 26, no. 1, pp. 12–19, 2011.
- [22] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.03300>
- [23] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024. [Online]. Available: <https://arxiv.org/abs/2412.15115>